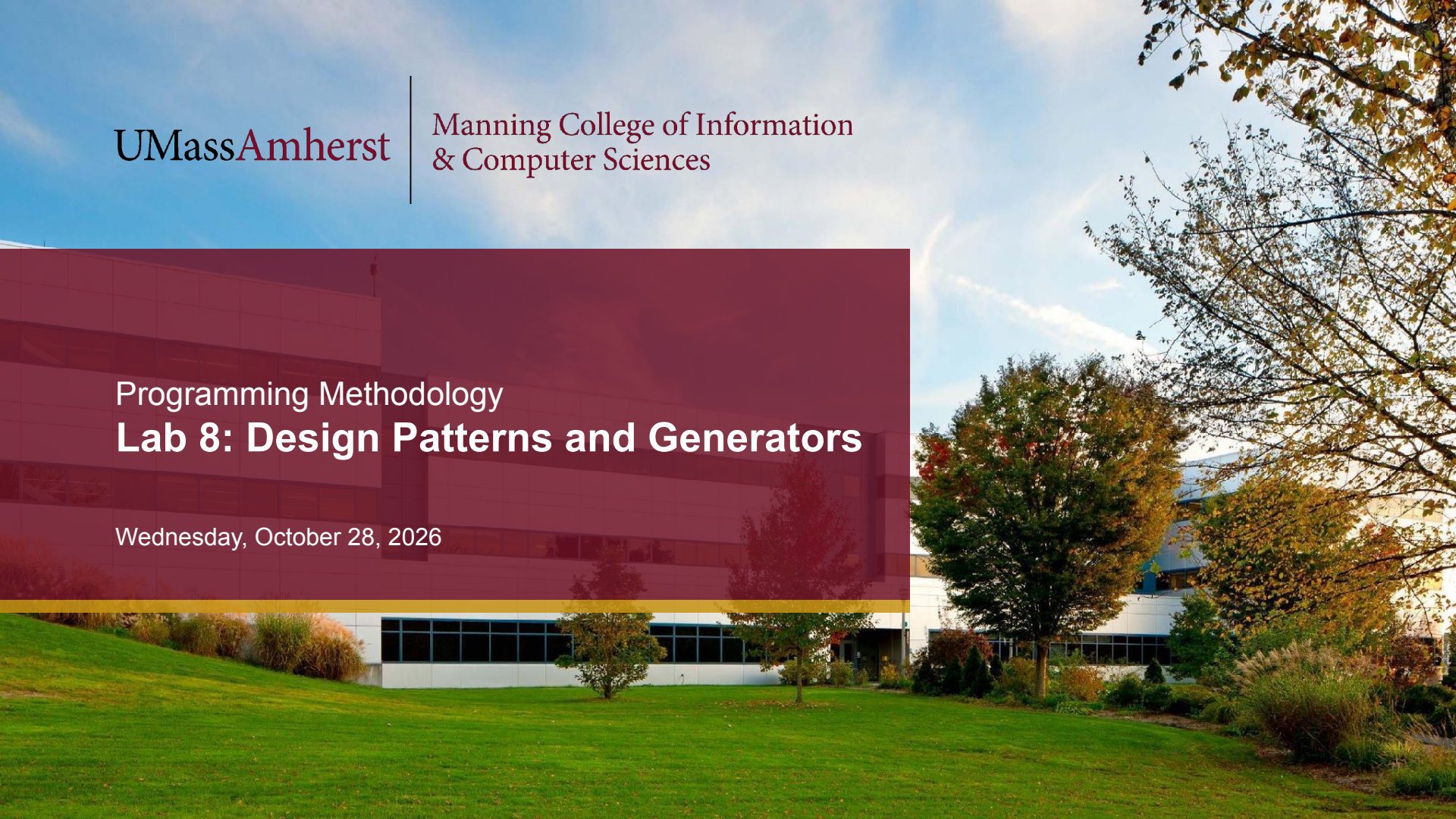


UMassAmherst | Manning College of Information & Computer Sciences

Programming Methodology

Lab 8: Design Patterns and Generators

Wednesday, October 28, 2026

Reminders

- **Midterm 2 is next Wednesday (11/4)**
  - **Time: 7-9 pm**
  - **Location: Mahar 108**

Reminders

- Homework 7a is due tonight (10/28) at 11:59 pm.
  - Come to office hours for help!
- If you need to miss lab for a valid and documented reason:
  - Fill out the excusal form on Canvas before the lab.
  - Waking up late or the bus being late are not excusable reasons.
- Submit what you have at the end of lab to Gradescope.
  - Lab submissions are due at 6:00 pm.
  - There are **two Gradescope assignments**: "Lab 8 Design Pattern Questions" and "Lab 8 Code". Both counts towards your lab grade.

Groups

If you aren't already, please sit with your new groups from last week!

The MVC exercise involves open-ended conceptual questions.

- You will be asked to talk to each other about them.

Singleton Review

The Singleton pattern ensures that a class has only one instance and provides a shared and controlled access point for that object.

Consider: When is shared state useful? What problems can it cause?

MVC Review

The key components of the **Model-View-Controller** design pattern are:

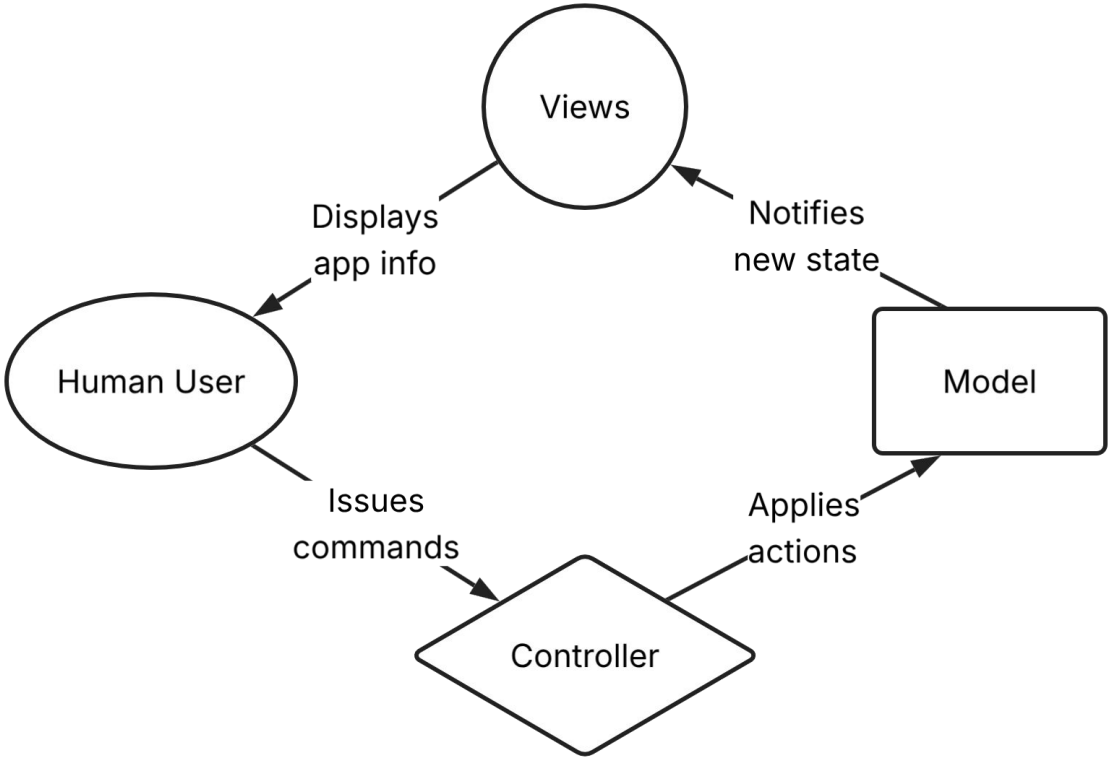
- **Model**: Encapsulates core data and logic of application
- **View**: Displays application state to the user
- **Controller**: Processes user commands and manages views

A key design principle of MVC:

- Application state and logic are *decoupled* from the user interface.

We won't review all of the details, because your first exercise ask you to talk about them!

MVC Review


Views
Displays
app info
Notifies
new state
Human User
Model
Issues
commands
Applies
actions
Controller

Exercise 1: Design Patterns

Discuss the following questions with your group and enter your answers individually into the Gradescope assignment titled "Lab 8 Design Pattern Questions".

1. What is a design pattern? Why might we use one?

2. What is the purpose of the Singleton pattern? What are its key characteristics?

3. Describe the nature of interactions between:
   - Model and View
   - Model and Controller
   - View and Controller

Generators Review

Generators allow us to produce values one at a time without manually tracking iterator state.

- `Generator` follows the `IterableIterator` interface.
- It's important to know how to create iterators and when to use them, but generators can be more concise and easier to work with.

Generators Review

Generators are created by **generator functions** that have the following syntax:

```
function* numbers(): Generator<number> {
  yield 1;
  yield 2;
  yield 3;
}

const gen = numbers();

gen.next().value // 1
gen.next().value // 2
gen.next().value // 3
gen.next().value // undefined
```

`function*` defines a generator function.

`yield` produces a value and pauses execution.

`.next()` resumes execution until the next `yield`.

When the function completes, `.next()` will return `{ done: true, value: undefined }`

The generator function takes care of making the `IteratorResult`s for you!

Exercise 2: Generators

Implement a generator function `mergedGenerator` that takes in two number iterators, each yielding values in non-decreasing order, and merges them into a single non-decreasing sequence.

Your solution should:

- Preserve duplicates and return all values from both iterators.
- Handle empty iterators and cases where one iterator finishes before the other.
- Use O(1) space.